PRACTICAL WORK

Discovering Arduino

Sensors, actuators and the automation of a laboratory step

Automated Laboratories · 27 October · 1 h 30 · Grove Beginner Kit for Arduino

In the previous weeks, you had to **establish the complete workflow of chemical and manual operations, with all the logic, and define the necessary equipment in terms of chemistry**. Today you will see how a small piece of it can be **automated**:

- what the operator *observes* is replaced by a **sensor**;
- what the operator *does* is replaced by an **actuator**;
- what the operator *decides* is replaced by a few lines of **program**.

You will work in groups on an Arduino-based kit. **Nothing can break and nothing has to be handed in**: the aim is to discover, experiment, and have fun. You will start with small exercises to get comfortable, then simulate a feature of your own process.

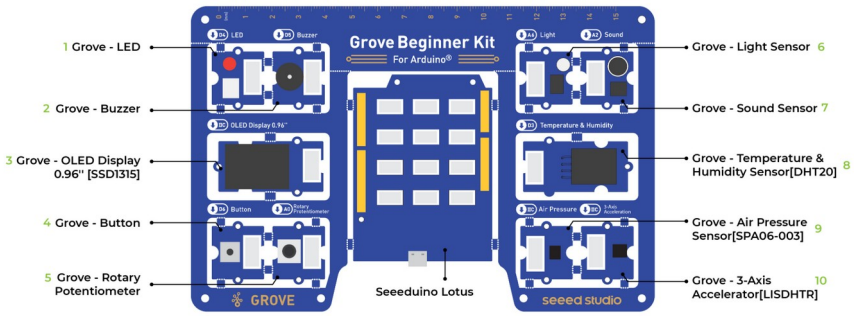
Roadmap (90 minutes)

Part	What you will do	Time
0. Getting started	Get the code from GitHub, set up the Arduino IDE, upload a first program	15 min
1. Button → LED	Run a provided program and play with it	10 min
2. Light sensor	**2a** see the measurements (Serial Monitor, Serial Plotter, screen); **2b** use a threshold to light the LED	20 min
3. Creative challenge	Invent your own small device using several sensors and actuators	15 min
4. Your lab process	Take a feature of your workflow, draw its automated version, and simulate it on the kit, with the help of an AI	30 min

0. Getting started

The kit

The Grove Beginner Kit is a single plate: all the sensors and actuators are already wired, so there is **no cable to connect**. The brain of the kit is the board in the middle (a *Seeeduino Lotus*, which is an Arduino Uno).



The Grove Beginner Kit. Pins are printed on the plate next to each module.

	On the board
Sensors (inputs)	Button · Potentiometer (rotary knob) · Light · Sound · Temperature & humidity · Air pressure · 3-axis accelerometer
Actuators (outputs)	LED · Buzzer · OLED screen

Set up your computer

1. Plug the kit into the computer with the USB cable.
2. Get the code from GitHub: **https://github.com/swisscatplus/Arduino_Grove.git**. Either **clone** it (`git clone https://github.com/swisscatplus/Arduino_Grove.git`), or **download** it (on the repository page: green **Code** button → **Download ZIP**, then extract it). The Arduino IDE needs the folder that contains `Arduino_Grove.ino` to be named exactly `Arduino_Grove`: rename it if you downloaded the ZIP (GitHub adds `-main`).
3. Open `Arduino_Grove.ino` in the Arduino IDE (**File → Open...**). A second tab, `sensors.h`, opens next to it: you never need to touch it.
4. Install the libraries (menu **Tools → Manage Libraries...**, search each name, click **Install**). Skip the ones that are already installed:
   - **U8g2** by oliver (OLED screen)
   - **DHT20** by Rob Tillaart (temperature and humidity)
   - **Grove - Barometer Sensor BMP280** (air pressure)
   - **Seeed Arduino LIS3DHTR** (accelerometer)
5. Select the board: **Tools → Board → Arduino Uno**, and the port: **Tools → Port** (the one that appears when the kit is plugged in). No port? The board probably needs a USB driver: see the box below.
6. Click **Upload** (the arrow at the top left). When it is done, the OLED screen shows **"Grove Kit / Ready!"**. You are ready.

No port in Tools → Port? Install the USB driver

The board of the kit uses a Silicon Labs **CP210x** USB chip, which needs a driver on most computers (administrator rights required). Go to https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=overview, open the **Downloads** tab, and download the driver for your system:

- **Windows 10 / 11:** *CP210x Universal Windows Driver*. Extract the ZIP completely (right-click → **Extract All**), open the extracted folder, right-click `silabser.inf` → **Install**, and accept the administrator prompt.
- **macOS:** *CP210x VCP Mac OSX Driver*. Open the ZIP, then the `.dmg`, and run the installer. If macOS blocks it, go to **System Settings → Privacy & Security**, click **Allow**, and restart the computer.

Then **unplug and replug the kit**. A port appears in **Tools → Port** (`COMx` on Windows, `/dev/cu...` on macOS). On Windows you can also check in **Device Manager → Ports**: *Silicon Labs CP210x USB to UART Bridge (COMx)*.

Still nothing? Try another USB cable (some are charge-only). Older kits (Seeeduino Lotus v1.0) use a CH340 chip and need the CH340 driver instead: ask the instructor.

How a program works

An Arduino program has two parts. `setup()` runs once at power-up (already done for you, do not touch it). `loop()` runs again and again, as long as the board is powered: this is where you write **your** code. Everything you write goes between the curly braces `{ }` of `loop()`.

The kit comes with simple functions you can call:

Function	Returns	Description
`setLED(true)` / `setLED(false)`	-	Turns the LED on / off
`buzz(frequency, duration_ms)`	-	Plays a sound, e.g. `buzz(1000, 200)`
`isButtonPressed()`	true / false	Is the button pressed right now?
`readPotentiometer()`	0 – 1023	Position of the knob
`readLight()`	0 – 1023	Light level
`readSound()`	0 – 1023	Sound level
`readTemperature()`	°C (decimal)	Air temperature
`readHumidity()`	% (decimal)	Air humidity
`readPressure()`	hPa (decimal)	Air pressure
`readAcceleration(x, y, z)`	-	Fills `x`, `y`, `z` (in g), declare them first: `float x, y, z;`
`displayText("line 1", "line 2", "line 3")`	-	Writes up to 3 lines on the OLED screen (16 characters per line, no accents)
`Serial.println(value)`	-	Sends a value to the computer (Serial Monitor / Plotter)

The complete list is in `README.md`.

1. Button → LED

> **Goal**
> Run a first program and see the full cycle of automation: the kit **senses** (button), **decides** (if...), and **acts** (LED).

Copy this code into `loop()` (replace the comment that is there), click **Upload**, and press the button on the kit.

```
void loop() {
  if (isButtonPressed()) {
    setLED(true);     // button pressed: LED on
  } else {
    setLED(false);    // otherwise: LED off
  }
}
```

Predict, then test

1. Replace `setLED(true);` with `buzz(1000, 200);`. What do you hear? Try other frequencies.
2. Make the LED turn on when the button is **not** pressed. (Hint: `!` means "not".)

> **Remember**
> A program is read from top to bottom, and `loop()` restarts as soon as it reaches the end: it checks the button thousands of times per second.

2. Light sensor

2a. See what the sensor measures (8 min)

> **Goal**
> Before you can decide at which light level something should happen, you need to **see** what the sensor measures. This is the same idea as a calibration in the lab.

Replace the content of `loop()` with:

```
void loop() {
  int light = readLight();
  Serial.println(light);                // send the value to the computer
  displayText("Light level:", String(light));   // and show it on the screen
  delay(100);                           // wait 0.1 s
}
```

1. Upload, then open the **Serial Monitor** (magnifying glass at the top right of the IDE). Set it to **115200 baud** (bottom right of the panel). You should see numbers scrolling.
2. Close it and open **Tools → Serial Plotter** (also 115200 baud). The values are now drawn as a live curve. Open only one of the two at a time.
3. Cover the light sensor with your hand, then shine a phone torch on it, and watch how the values change.

2b. Act: switch the LED on when it gets dark (12 min)

> **Goal**
> Use a **threshold** to turn a measurement into a decision: when the light drops below a chosen value, the LED switches on.

Replace your loop() with the code below (the const line goes just above it).

```
const int THRESHOLD = 0;      // the LED turns on below this light level

void loop() {
 int light = readLight();

 if (light < THRESHOLD) {
  setLED(true);
 } else {
  setLED(false);
 }

 // Plot the light and the threshold together
 Serial.print("Light:");
 Serial.print(light);
 Serial.print(",Threshold:");
 Serial.println(THRESHOLD);
 delay(100);
}
```

1. Upload, open the **Serial Plotter** and cover / uncover the sensor: you see the light curve and the threshold line.
2. **Change** THRESHOLD and upload again, until the LED switches on at the light level you want (for example when you partly cover the sensor). The LED switches at the moment the light curve crosses the threshold line.

If you have time

- Also make the **buzzer** beep when it gets dark.
- Put the threshold right at the level of the room light. What happens when the light hovers around it? Why would that be a problem in a real experiment?
- Replace THRESHOLD by readPotentiometer() so that you tune the threshold by turning the knob, and watch it move on the plot.
- Show the word "DARK" or "BRIGHT" on the screen with displayText(...).

3. Creative challenge

> **Goal**
> Be creative: invent a small device that uses **several sensors and actuators together**. Time limit: 15 minutes, so keep it simple. If you are not finished, that is fine: move on to part 4.

Your device must...

- use **at least 2 sensors** (inputs) and **at least 2 actuators** (outputs, the screen counts);
- take **at least one decision** based on a threshold;
- have **one value adjustable with the potentiometer**.

How to proceed

1. **Pick an idea** from the list below, or invent your own (2 min).
2. **Sketch the flowchart on paper** (3 min): rectangles for actions and measurements, a diamond for each decision, as in the examples of part 4.

3. **Write the code** (10 min), by yourself or with the help of an AI (see "Using an AI" at the end of this document).

Idea	Inputs	Outputs	Twist
Breath detector	Humidity	LED, screen	Blow on the sensor: the LED lights when the humidity jumps above your threshold.
Clap switch	Sound	LED	One clap switches the LED on, the next one off. (You need a variable that remembers the state.)
Dark-box alarm	Light, button	Buzzer, LED	A light-sensitive sample is in a box. Alarm when someone opens it; the button resets the alarm.
Shake counter	Accelerometer, button	Screen	Count how many times the kit is shaken. The button resets the counter.
Fever alarm	Temperature, potentiometer	LED, buzzer, screen	The knob sets the alarm temperature. Warm the sensor with your finger.
Incubation timer	Button, potentiometer	Screen, buzzer	The knob sets a duration, the button starts it, the buzzer rings at the end. (Hint: `millis()`.)

4. Your lab process

> **Goal**
> Take **one feature** of the manual workflow you built in the previous weeks, imagine how it could be **automated**, and **simulate** it on the kit. You will use an AI assistant to turn a photo of your workflow into code.

Step 1: choose one feature (5 min)

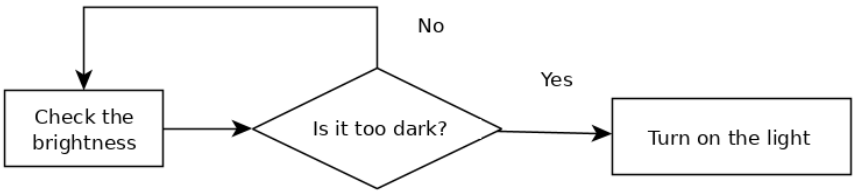
Do not try to automate the whole process. Pick **one small piece** that contains a measurement, a decision, and an action. For example: *"check the temperature; if it is too high, stop the heating"*, or *"detect the color change; when it happens, stop adding reagent"*.

Step 2: draw the automated workflow (10 min)

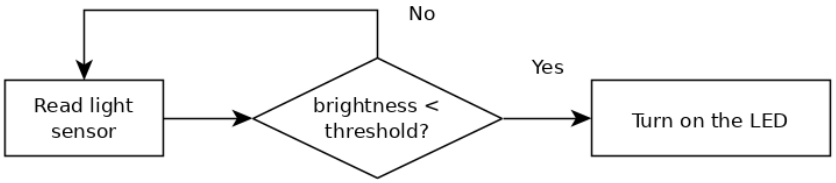
On paper, redraw that feature as a flowchart, once with the operator (manual version, if it helps) and once **automated**. Go through each box and ask:

- what the operator **observes** → which **sensor** could do it?
- what the operator **does** → which **actuator** could do it?
- what the operator **decides** → which **threshold or rule** in the program?

Write the kit component you choose in each box. Example, for a system that switches on a light when it gets too dark:



Manual workflow: the operator checks the brightness and decides.



Automated workflow: a sensor, a threshold, an actuator.

From the real lab to the kit

The kit cannot heat a flask or read a real pH, but it can **stand in** for them. Use this table to choose your stand-ins:

In the real lab	On the kit
Operator confirms / starts a step	Button
Setpoint (target temperature, volume...)	Potentiometer (turn the knob)
Temperature probe	Temperature sensor (warm it with a finger)
pH probe, or any other analog probe	Potentiometer (turn it to "change the pH")
Color change, turbidity	Light sensor (cover it, or use a phone torch)
Vapor, evaporation, humidity	Humidity sensor (breathe on it)
Stirring, shaking, a sample being moved	Accelerometer (shake the kit)
Something happening in the room (door, noise)	Sound sensor (clap)
Heater, pump, valve, stirrer being switched on	LED
Alarm	Buzzer
Display of the values / log of the experiment	OLED screen and Serial Monitor

Step 3: let the AI write the code (10-12 min)

1. Take a **photo of your automated flowchart** (with your phone, and send it to your computer, or use the AI assistant on your phone).
2. Open the AI assistant of your choice. Follow PROMPT_AI.md: paste **Message 1** (it teaches the AI the functions of the kit), then send **Message 3** with your photo attached.

3. The AI first gives a table: *which component stands for which box, and how to trigger it*. **Read it and check it makes sense** for your workflow before using the code.
4. Copy the complete file it gives you into the Arduino IDE (**Ctrl+A**, **Ctrl+V**), **Upload**, and play your process by triggering the sensors by hand.

> **No AI? You can code by hand.**
> Using an AI is a choice, not an obligation. You can write the code yourself with what you learned in parts 1 to 3. Be aware that it is usually slower, so keep your feature very small.

Step 4: test, tweak, show (5 min)

- Does the simulation behave like your flowchart? If not, tell the AI what you see ("the LED stays on", "it is too sensitive") and ask for the complete file again.
- Adjust a threshold. Add a second feature (a buzzer alarm, a message on the screen...).
- Show your simulation to a neighboring group, and look at theirs.

> **Think about it (no need to write anything)**
> - What does the kit **simulate**, and what would you need in the **real lab** (a real probe, a relay for a heater, a safety limit...)?
> - Which part of your process is the **hardest to automate**? Why?
> - What did the sensor *not* see that a human operator would have noticed?

Using an AI to write the code

An AI assistant (ChatGPT, Claude, Gemini, Copilot...) can help you to:

- generate a first version of the program;
- explain how a piece of code works;
- change thresholds, messages or behaviors;
- find and fix errors: copy the error message from the IDE into the chat.

But it **does not know your kit**, and it can invent functions or use the wrong pins. This is why PROMPT_AI.md gives it the list of functions first. Always **test** the program, and make sure you **understand** what it does: ask the AI to explain if you do not.

A good request is specific

Vague	Specific
"Make a temperature alarm."	"When the temperature is above the value set with the potentiometer (between 20 and 40 °C), switch the LED on and beep every 2 seconds. Show the temperature and the limit on the screen."
"It does not work."	"The LED stays on all the time, even when I cover the sensor. The Serial Monitor shows values between 80 and 600."

Troubleshooting

Problem	What to try
No port in **Tools → Port**	Install the **Silicon Labs CP210x USB driver**: see the box "No port in Tools → Port?" in part 0. Otherwise try another USB cable (some are charge-only) and ask the instructor.
Error `... was not declared in this scope`	A typo in a function or variable name (capital letters matter). Or a `{` or `}` is missing.
Error `fatal error: ... .h: No such file`	A library is missing: install it (Tools → Manage Libraries).
Upload worked but nothing changed	Did you click **Upload** after your last change? Check that the board and port are still selected.
Serial Monitor shows strange symbols	Set it to **115200 baud** (bottom right of the panel).
Serial Plotter shows nothing	Close the Serial Monitor (only one at a time), and check 115200 baud.
Message "BMP280 pressure sensor not found"	Only the pressure measurement is affected. Ignore it unless you use `readPressure()`.
Temperature / humidity look wrong	Ask the instructor: the kit may have a different temperature sensor.

Stuck? Ask the instructor, or ask a neighboring group. Enjoy!